

# Audio Analysis Cheatsheet — Test 2 Prep

---

## ESSENTIAL SETUP

```
library(tuneR)
library(seewave)
```

## CORE CODE (MOST IMPORTANT — MEMORISE THIS)

```
# — THE FREQUENCY SPECTRUM PLOTTING FUNCTION —————
plot.frequency.spectrum <- function(X.k, xlims=c(0,length(X.k))) {
  plot.data <- cbind(0:(length(X.k)-1), Mod(X.k))
  plot.data[2:length(X.k),2] <- 2*plot.data[2:length(X.k),2] # scale (all except
DC bin)
  plot(plot.data, t="h", lwd=2, main="",
        xlab="Frequency (Hz)", ylab="Strength",
        xlim=xlims, ylim=c(0,max(Mod(plot.data[,2]))))
}

# — LOAD & PLAY —————
z <- readWave("path/to/file.wav")
play(z)

# — DETECT SOUND ACTIVITY (TIMER) —————
timer(z, f=22050, threshold=20, msmooth=c(50,0))
# f          = sampling rate (check z@samp.rate if unsure)
# threshold = energy % threshold to count as "sound"
# msmooth   = smoothing window

# — FFT & FREQUENCY PLOTS —————
Zk <- fft(z@left) # FFT on LEFT channel
plot.frequency.spectrum(Zk) # full spectrum (will be mirrored)
plot.frequency.spectrum(Zk[1:20000]) # zoom into first 20000 bins

# — SPECTROGRAM & AVERAGE SPECTRUM —————
layout(t(1:1)) # reset layout before dynspec
dynspec(z, wl=1024, osc=T) # dynamic spectrogram (STFT) with
oscillogram
spectro(z) # static spectrogram
meanspec(z) # average spectrum across whole duration
```

## WAVE OBJECT STRUCTURE

| Slot                     | Description                             | Example                          |
|--------------------------|-----------------------------------------|----------------------------------|
| <code>z@left</code>      | Left channel samples (use this for FFT) | <code>fft(z@left)</code>         |
| <code>z@right</code>     | Right channel samples                   | <code>fft(z@right)</code>        |
| <code>z@samp.rate</code> | Sampling rate in Hz                     | <code>z@samp.rate</code> → 22050 |
| <code>z@bit</code>       | Bit depth                               | <code>z@bit</code> → 16          |

**FFT output is mirrored** — only the first half is meaningful.

Use `Zk[1:(length(Zk)/2)]` to avoid the mirror.

## KEY FUNCTIONS QUICK REFERENCE

```

readWave("file.wav")           # load WAV
readMP3("file.mp3")           # load MP3 (tuneR)
writeWave(z, "output.wav")     # save WAV

extractWave(z, from=2, to=5, xunit="time") # cut by time (seconds)
extractWave(z, from=1000, to=5000, xunit="samples") # cut by sample index

normalize(wave, unit="16")      # normalize to 16-bit
bind(w1, w2)                   # concatenate waves

sine(440, duration=44100)       # synthetic sine wave
noise(kind="pink", duration=44100) # noise
pulse(220, duration=44100)      # pulse wave
sawtooth(100, duration=44100)   # sawtooth wave
square(200, duration=44100)     # square wave

fft(x)                          # Fast Fourier Transform
Mod(fft(x))                     # magnitude of FFT

```

## MANUALLY FIND PEAK FREQUENCY & AMPLITUDE

The `timer()` function shows WHEN sound occurs but NOT the peak frequency/amplitude.

Use the code below to find peak values yourself.

```

Zk <- fft(z@left)

# Use only first half (FFT is symmetric/mirrored)
half_n <- floor(length(Zk) / 2)
mags <- Mod(Zk[1:half_n])
mags[2:length(mags)] <- 2 * mags[2:length(mags)] # same scaling as plot function

# Find peak
peak_bin <- which.max(mags) # bin index (1-based)

```

```

peak_amp <- mags[peak_bin] # amplitude (strength)
peak_freq <- (peak_bin - 1) * z@samp.rate / length(Zk) # convert bin → Hz

cat("Peak Frequency :", round(peak_freq, 2), "Hz\n")
cat("Peak Amplitude :", round(peak_amp, 2), "\n")
cat("At bin index   :", peak_bin, "\n")

```

---

## CONVERT BIN INDEX → ACTUAL Hz

```
Frequency (Hz) = (bin_index - 1) × samp.rate / N
```

where  $N = \text{length}(\text{fft\_output})$  and  $\text{bin\_index}$  is 1-based (R indexing).

---



---

## EXAMPLE EXAM QUESTIONS & MODEL ANSWERS

---

### Q1 — Load a WAV file, perform FFT, and plot the frequency spectrum

**Question:** Load `speech.wav`, apply FFT on the left channel, and plot the frequency spectrum zoomed to the first 15000 bins.

```

library(tuneR)
library(seewave)

plot.frequency.spectrum <- function(X.k, xlims=c(0,length(X.k))) {
  plot.data <- cbind(0:(length(X.k)-1), Mod(X.k))
  plot.data[2:length(X.k),2] <- 2*plot.data[2:length(X.k),2]
  plot(plot.data, t="h", lwd=2, main="",
        xlab="Frequency (Hz)", ylab="Strength",
        xlim=xlims, ylim=c(0,max(Mod(plot.data[,2]))))
}

z <- readWave("speech.wav")
Zk <- fft(z@left)
plot.frequency.spectrum(Zk[1:15000])

```

#### Key points:

- Always use `z@left` for FFT, not `z` directly
  - Zoom using `Zk[1:N]` to see detail — full spectrum is mirrored
  - The function scales all bins except bin 1 (DC) by  $\times 2$
-

## Q2 — Load an MP3 file

**Question:** Load `music.mp3`, inspect its properties, and play it.

```
library(tuneR)

z <- readMP3("music.mp3")
play(z)

# Inspect properties
z@samp.rate # sampling rate
z@bit       # bit depth
length(z@left) / z@samp.rate # duration in seconds
```

**Key trap:** `readWave()` will FAIL on MP3 — you must use `readMP3()`.

---

## Q3 — Load audio from an MP4 / other format

**Question:** You are given `interview.mp4`. How do you load and analyse it in R?

```
# Step 1: convert MP4 to WAV using the av package
install.packages("av")
library(av)
av_audio_convert("interview.mp4", "interview_audio.wav")

# Step 2: load the extracted WAV normally
library(tuneR)
z <- readWave("interview_audio.wav")
play(z)

# Step 3: proceed with FFT as normal
Zk <- fft(z@left)
plot.frequency.spectrum(Zk[1:(length(Zk)/2)])
```

**Key trap:** R cannot read MP4 directly. Always convert first.

Other formats (OGG, FLAC) also need conversion via `av_audio_convert()`.

---

## Q4 — Cut / extract a specific time segment

**Question:** From `radio.wav` (1 minute long), extract the segment from **10 seconds to 25 seconds** and analyse only that portion.

```
library(tuneR)
library(seewave)

z <- readWave("radio.wav")
```

```
# Method 1: extractWave (cleanest)
z_cut <- extractWave(z, from=10, to=25, xunit="time")

# Method 2: manual sample indexing
sr      <- z@samp.rate
start_samp <- 10 * sr  # sample at 10 s
end_samp  <- 25 * sr  # sample at 25 s
z_cut     <- z[start_samp:end_samp]

# Analyse the cut segment
play(z_cut)
Zk_cut <- fft(z_cut@left)
plot.frequency.spectrum(Zk_cut[1:(length(Zk_cut)/2)])
spectro(z_cut)
```

**Key trap:** Always multiply seconds by `z@samp.rate` to get sample index.

Using `xunit="samples"` instead of `xunit="time"` if you know sample numbers.

## Q5 — Find peak frequency, peak amplitude, and where it occurred

**Question:** For `babycry.wav`, find the dominant frequency, its amplitude, and at which time (in seconds) the loudest moment occurs.

```
library(tuneR)
library(seewave)

z <- readWave("babycry.wav")
Zk <- fft(z@left)

# — Peak Frequency & Amplitude —————
half_n <- floor(length(Zk) / 2)
mags <- Mod(Zk[1:half_n])
mags[2:length(mags)] <- 2 * mags[2:length(mags)]

peak_bin <- which.max(mags)
peak_freq <- (peak_bin - 1) * z@samp.rate / length(Zk)
peak_amp <- mags[peak_bin]

cat("Peak Frequency:", round(peak_freq, 2), "Hz\n")
cat("Peak Amplitude:", round(peak_amp, 2), "\n")

# — When (time) does the loudest moment occur? —————
# Use a short-window energy envelope
samples <- z@left
win_size <- z@samp.rate # 1-second windows
n_windows <- floor(length(samples) / win_size)
energies <- sapply(1:n_windows, function(i) {
  seg <- samples[((i-1)*win_size + 1):(i*win_size)]
  sum(seg^2)
```

```

}))
loudest_window <- which.max(energies)
cat("Loudest at ~", loudest_window, "seconds\n")

# — Visual confirmation —————
timer(z, f=z@samp.rate, threshold=20, msmooth=c(50,0))
spectro(z)

```

## Q6 — Use timer() to detect sound activity

**Question:** Use `timer()` on `tico` data to identify when the bird is singing vs silent.

```

library(tuneR)
library(seewave)

data(tico)

# threshold: % of max energy below which = silence
# msmooth: smoothing to reduce noise in the detection
timer(tico, f=22050, threshold=5, msmooth=c(50,0))

# If using your own wav:
# z <- readWave("file.wav")
# timer(z, f=z@samp.rate, threshold=20, msmooth=c(50,0))

```

**What timer() shows:**

- Blue = sound detected
- White = silence
- Bottom axis = time in seconds

**Key trap:** `f` must match the actual sampling rate of the file — use `z@samp.rate`, not a hardcoded number unless you are sure.

## Q7 — Compare two audio files using spectrogram and mean spectrum

**Question:** Compare `voice_before.wav` and `voice_after.wav` side by side using spectrograms and average spectra.

```

library(tuneR)
library(seewave)

z1 <- readWave("voice_before.wav")
z2 <- readWave("voice_after.wav")

# Side-by-side spectrograms
par(mfrow=c(1,2))

```

```
spectro(z1, main="Before")
spectro(z2, main="After")

# Side-by-side mean spectra
par(mfrow=c(2,1))
meanspec(z1, main="Before")
meanspec(z2, main="After")

# FFT comparison
layout(t(1:1))
Zk1 <- fft(z1@left)
Zk2 <- fft(z2@left)
par(mfrow=c(2,1))
plot.frequency.spectrum(Zk1[1:20000])
title("Before")
plot.frequency.spectrum(Zk2[1:20000])
title("After")
```

## Q8 — Create a synthetic wave and verify its frequency via FFT

**Question:** Create a 440 Hz sine wave at 8000 Hz sampling rate, then verify using FFT that the dominant frequency is indeed ~440 Hz.

```
library(tuner)
library(seewave)

plot.frequency.spectrum <- function(X.k, xlimits=c(0,length(X.k))) {
  plot.data <- cbind(0:(length(X.k)-1), Mod(X.k))
  plot.data[2:length(X.k),2] <- 2*plot.data[2:length(X.k),2]
  plot(plot.data, t="h", lwd=2, main="",
        xlab="Frequency (Hz)", ylab="Strength",
        xlim=xlimits, ylim=c(0,max(Mod(plot.data[,2]))))
}

sr <- 8000
t <- seq(0, 2, 1/sr)
y <- (2^15 - 1) * sin(2*pi*440*t)

Yk <- fft(y)
plot.frequency.spectrum(Yk[1:1000]) # zoom to see the spike

# Manually confirm peak
mags <- Mod(Yk[1:floor(length(Yk)/2)])
peak_bin <- which.max(mags)
peak_hz <- (peak_bin - 1) * sr / length(Yk)
cat("Detected frequency:", round(peak_hz, 1), "Hz\n") # should be ~440
```

## Q9 — Analyse only a specific frequency band

**Question:** For `music.wav`, show only the frequency content between 0 and 5000 Hz.

```
library(tuneR)
library(seewave)

z <- readWave("music.wav")
Zk <- fft(z@left)

# How many bins fit in 5000 Hz?
# bin_count = 5000 / (samp.rate / N) = 5000 * N / samp.rate
N <- length(Zk)
bin_limit <- round(5000 * N / z@samp.rate)

plot.frequency.spectrum(Zk[1:bin_limit],
                        xlim=c(0, bin_limit))
```

**Formula to remember:**

```
bin_limit = target_Hz * N / samp.rate
```

## Q10 — Full pipeline: load → cut → FFT → find peak → plot spectrogram

**Question:** Given `podcast.wav`, extract the first 30 seconds, find the dominant frequency, and produce a spectrogram.

```
library(tuneR)
library(seewave)

plot.frequency.spectrum <- function(X.k, xlim=c(0,length(X.k))) {
  plot.data <- cbind(0:(length(X.k)-1), Mod(X.k))
  plot.data[2:length(X.k),2] <- 2*plot.data[2:length(X.k),2]
  plot(plot.data, t="h", lwd=2, main="",
        xlab="Frequency (Hz)", ylab="Strength",
        xlim=xlim, ylim=c(0,max(Mod(plot.data[,2]))))
}

# 1. Load
z <- readWave("podcast.wav")
cat("Duration:", length(z@left)/z@samp.rate, "seconds | Rate:", z@samp.rate,
    "Hz\n")

# 2. Cut first 30 seconds
z30 <- extractWave(z, from=0, to=30, xunit="time")

# 3. FFT
Zk <- fft(z30@left)
```



```
# 4. Find peak
half_n <- floor(length(Zk)/2)
mags <- Mod(Zk[1:half_n])
mags[2:length(mags)] <- 2 * mags[2:length(mags)]
peak_bin <- which.max(mags)
peak_freq <- (peak_bin - 1) * z30@samp.rate / length(Zk)
cat("Dominant frequency:", round(peak_freq, 1), "Hz\n")

# 5. Plot
layout(t(1:1))
plot.frequency.spectrum(Zk[1:half_n])
spectro(z30)
meanspec(z30)
```

## COMMON TRAPS TO AVOID

| Trap                                                                      | What goes wrong                    | Fix                                              |
|---------------------------------------------------------------------------|------------------------------------|--------------------------------------------------|
| Using <code>z</code> instead of <code>z@left</code> in <code>fft()</code> | Error — fft needs a numeric vector | Always <code>fft(z@left)</code>                  |
| Using <code>readWave()</code> on an MP3                                   | Error — wrong reader               | Use <code>readMP3()</code>                       |
| Not converting MP4 first                                                  | Error — unsupported format         | <code>av_audio_convert()</code> first            |
| Plotting full FFT (mirrored)                                              | Confusing double-peaked plot       | Use <code>Zk[1:(length(Zk)/2)]</code>            |
| Wrong <code>f=</code> in <code>timer()</code>                             | Timer misdetects silence/sound     | Use <code>z@samp.rate</code> , not hardcoded     |
| Forgetting <code>layout(t(1:1))</code> before <code>dynspec()</code>      | Plot layout broken                 | Always reset layout first                        |
| Using sample index without multiplying by <code>samp.rate</code>          | Wrong time position                | <code>time_s * z@samp.rate = sample index</code> |